

torch_vn_algebra: batched random operators, states, channels and constrained optimisation in finite-dimensional Type I von Neumann algebras

Irina Nikolaeva

Andrej Novikov

Abstract

`torch_vn_algebra` is a PyTorch library for numerical work in algebras $\mathcal{M} = \bigoplus_{c=1}^C M_{k_c}(\mathbb{C})$, the finite-dimensional Type I von Neumann algebras that describe quantum systems with superselection rules. Operators, states and channels are stored as batched, block-diagonal tensors, so that thousands of random samples, or thousands of independent starts of an optimisation, are processed at once on a CPU or a GPU. The library provides random operators with prescribed spectra and Haar-distributed eigenvectors, the three trace functionals that are natural on $\mathcal{M}$, states and Lüders conditioning, completely positive maps within and between sectors, plain and charge-conserving (fused) tensor products with partial traces, spin chains with a conserved magnetisation and momentum, sparse Lanczos and Krylov solvers for large sectors, Hamiltonian and Lindblad dynamics (including jumps between sectors), differentiable parametrisations with a prescribed Michelson contrast, and safeguards that estimate run time and memory before expensive steps. We describe the design, show how the library is used on short complete examples, and validate it against exact results from random matrix theory, quantum information and integrable spin chains (free-fermion spectra and dynamics, Heisenberg ring energies, Peschel’s entanglement formula, the GOE–Poisson crossover of disordered chains). Applied to the trace inequalities of our earlier work, constrained optimisation replaces Monte Carlo sampling: it recovers the sharp bounds that sampling misses in all but the smallest dimensions.

Program summary

Program title: `torch_vn_algebra`

Developer’s repository link: https://github.com/andre-a-no/von_neumann_type_i

Licensing provisions: MIT

Programming language: Python (≥ 3.9), PyTorch (≥ 1.9 , tested with 2.x)

Nature of problem: Numerical experiments in finite-dimensional Type I von Neumann algebras $\bigoplus_c M_{k_c}(\mathbb{C})$, which describe quantum systems with superselection rules or conserved charges: random operators with prescribed spectra, several trace functionals, states, channels within and between sectors, symmetric tensor products, spin chains, Hamiltonian and Lindblad dynamics, and extremal problems for trace inequalities under spectral constraints.

Solution method: Batched block-diagonal tensors processed by PyTorch linear algebra on CPUs and GPUs; exact Haar sampling; spectral functional calculus; Kraus representations; Lanczos and Krylov methods for sparse sector Hamiltonians; differentiable parametrisations with batched multistart gradient optimisation; run-time and memory estimates before expensive steps.

Additional comments: more than 250 unit tests, run in continuous integration; every table and figure of this paper is regenerated by `scripts/paper/run_all.sh`.

1 Introduction

Where block-diagonal algebras arise. Whenever a conserved quantity, a symmetry or a superselection rule splits a Hilbert space into sectors, the physically relevant operators are block diagonal. Their algebra is a finite-dimensional Type I von Neumann algebra

$$\mathcal{M} = \bigoplus_{c=1}^C M_{n_c}(\mathbb{C}) \quad \text{acting on} \quad \mathcal{H} = \bigoplus_{c=1}^C \mathbb{C}^{k_c}, \quad k_c \leq n_c, \quad (1)$$

where C is the number of sectors (direct summands), $M_n(\mathbb{C})$ the algebra of complex $n \times n$ matrices, and k_c the dimension of the subspace on which the c -th factor acts in a given computation (in most applications $k_c = n_c$). Every finite-dimensional von Neumann algebra has this form [18], and its centre is spanned by the sector projections.

From a symmetry to the algebra. The link between physics and (1) is the commutant. If a group G (or a set of conserved quantities) acts unitarily on $\mathcal{H}$ and $\mathcal{H} \cong \bigoplus_{\lambda} V_{\lambda} \otimes \mathbb{C}^{m_{\lambda}}$ is its decomposition into irreducible representations V_{λ} with multiplicities m_{λ} , then the operators commuting with the action form the algebra $\bigoplus_{\lambda} \mathbb{K}_{V_{\lambda}} \otimes M_{m_{\lambda}}(\mathbb{C}) \cong \bigoplus_{\lambda} M_{m_{\lambda}}(\mathbb{C})$. For one conserved self-adjoint charge Q (particle number, magnetisation, Hamming weight) the irreducibles are one-dimensional and m_{λ} is the dimension of the eigenspace of Q with eigenvalue λ . A Hamiltonian that conserves Q , a time evolution generated by it, and every observable compatible with a superselection rule for Q lie in this commutant; the sectors are the eigenvalues of Q , and the centre consists of the functions of Q . Physical modelling with a symmetry is therefore computation in an algebra of the form (1).

Examples are abundant:

- superselection rules for charge, parity or particle number [19, 20] and the classical-quantum structure produced by decoherence and einselection [21];
- many-body Hamiltonians with conserved charges, whose exact diagonalisation and dynamics are carried out sector by sector [22, 23], and symmetry-resolved entanglement [24, 25];
- quantum information: decoherence-free subspaces and noiseless subsystems [26, 27], operator-algebra quantum error correction [28], and quantum algorithms that preserve a conserved quantity, such as the XY mixers of the quantum alternating operator ansatz, which keep the Hamming weight [29];
- random matrix theory, where symmetries decompose ensembles into blocks and spectral statistics are meaningful only inside a block [30, 12];
- machine learning: by Schur’s lemma the linear maps that commute with a group representation (equivariant layers) form exactly an algebra of the form (1), with n_c the multiplicities of the irreducible representations [31, 32]; random-matrix methods are used to analyse trained weights [33];
- operator theory, our original motivation: inequalities between traces of products characterise the trace and the centre of an operator algebra [2, 3, 4, 5], and their failure can be quantified by spectral quantities such as the Michelson contrast [6, 7].

On $\mathcal{M}$ the trace is not unique: every positive weighting of the sectors gives one, and statements such as “trace preserving” or “the dual map” depend on that choice as soon as maps change the sector structure.

What existing software offers. Table 1 compares the features relevant here. Dense and sparse linear algebra (NumPy/SciPy [34, 35], with ARPACK [36] for sparse eigenproblems) handles single matrices; QuTiP [8] and the quantum-information module of Qiskit [9] represent states, operators and channels on one Hilbert space; **dynamiqs** [10] adds batching, GPUs and automatic differentiation for Schrödinger and Lindblad dynamics; QuSpin [23] builds symmetry-reduced bases for exact diagonalisation; tensor-network libraries (ITensor [37], TeNPy [38]) store charge-conserving tensors in block-sparse form; RTNI [39] integrates Haar-random tensor networks symbolically. None of them treats the algebra (1) itself, with its centre, several traces, states and maps between algebras of different sector structure, as the basic object, and none combines it with batched sampling and optimisation over operators with prescribed spectra.

	NumPy/ SciPy	QuTiP	Qiskit q. info	dynamiqs	QuSpin	ITensor/ TeNPy	this work
GPU execution	–	(✓)	–	✓	–	(✓)	✓
batched over many operators	(✓)	–	–	✓	–	–	✓
automatic differentiation	–	(✓)	–	✓	–	(✓)	✓
symmetry sectors of many-body systems	–	–	–	–	✓	✓	✓
algebra $\bigoplus_c M_{n_c}$ with centre and several traces	–	–	–	–	–	–	✓
channels within and between sectors	–	(✓)	(✓)	–	–	–	✓
prescribed spectra, Haar/COE/CSE per sector	(✓)	(✓)	(✓)	–	–	–	✓
optimisation under spectral constraints	–	–	–	–	–	–	✓
large single systems	✓	✓	✓	✓	✓	✓	(✓)

Table 1: Features relevant to computations in the algebra (1), to the best of our knowledge at the time of writing (✓: supported; (✓): partly, through plug-ins, or with user code; –: not supported). The last row is where the specialised packages are the better choice.

Why not one large matrix? Nothing forces one to treat $\mathcal{M}$ as a separate object: it embeds into $M_N(\mathbb{C})$, $N = \sum_c k_c$, as the block-diagonal matrices, and every package above handles a single $N \times N$ matrix (the case $C = 1$ of (1)). The embedding is exact for linear algebra, so the question is only what it costs and what has to be supplied by hand.

- *Cost.* A product, an inverse or a spectral decomposition in M_N costs $O(N^3)$ time and $O(N^2)$ memory, in $\mathcal{M}$ it costs $\sum_c k_c^3$ and $\sum_c k_c^2$. For C equal blocks this is a factor C^2 in time and C in memory; for the half-filling sector of a chain of $L = 16$ spins ($N = 2^{16}$, largest block 12870) the dense M_N computation is out of reach, the block computation is routine.
- *Measures.* The unitary group of $\mathcal{M}$ is $\prod_c U(k_c)$, a subgroup of $U(N)$ of measure zero. A Haar-random element of $U(N)$ is not in $\mathcal{M}$, and a random element of $\mathcal{M}$ has to be produced block by block. The same holds for random states, for the circular ensembles and for any optimisation: an unconstrained gradient step in M_N leaves $\mathcal{M}$.
- *Traces.* On M_N the trace is unique up to a factor; on $\mathcal{M}$ the faithful traces form a C -parameter family $\sum_c w_c \text{Tr}_c$. Densities, “trace preserving” and the dual of a map between algebras with different sectors depend on the choice of the weights; on the embedded matrices this information is no longer visible.
- *Centre.* The centre of $\mathcal{M}$ (the sector projections) is what physicists call the classical or superselected part of the system; conditioning on a sector, post-selection and sector-resolved quantities are operations with the centre, and they have to be reconstructed from the block indices when $\mathcal{M}$ is stored as one matrix.

The library keeps these four pieces of structure inside the data type. For $C = 1$ it reduces to batched linear algebra in $M_n(\mathbb{C})$, and all its functions accept that case. The first version of this library [1] shows that the bookkeeping is error-prone: its complex Haar sampler was not Haar distributed, its circular ensembles were not the circular ensembles, and several conclusions drawn from Monte Carlo samples were wrong; all of this is corrected and tested here.

Challenges. Turning the algebra into an efficient and reliable tool raises concrete problems, each addressed by a design decision described in Section 3: (i) heterogeneous block sizes have to be mapped onto batched GPU kernels; (ii) trace conventions and dual maps must stay consistent when maps change the sectors; (iii) random ensembles must be exactly Haar (or COE/CSE) in every block; (iv) gradients of spectral functionals are singular at degenerate spectra, exactly where extremal operators tend to lie; (v) extremal values sit on sets of tiny measure that random sampling does not reach in higher dimensions (Section 6); (vi) batched workloads easily exceed the time or memory budget of a device; (vii) the number field (real or complex) and the precision change both cost and results (Section 3).

Objective and scope. `torch_vn_algebra` is a library for numerical experiments *in* the algebra (1): sampling and optimising operators, states and channels, evaluating trace and spectral functionals, and formulating physical models (spin chains, open systems, noisy symmetric quantum circuits) whose symmetry is encoded in the sector structure. It is not a circuit simulator and not a tensor-network code, and for single large systems the specialised packages above remain the better choice; Section 8 quantifies where the library gains and where it does not.

Contributions and outline. Section 2 fixes notation and conventions. Section 3 describes the software stack, the workflow, how the challenges (i)–(vii) are met, and the choice of number field and precision. Section 4 shows the library on short complete programs, including a noisy quantum circuit. Section 5 validates it against exact results and checks the trace functionals. Section 6 replaces Monte Carlo sampling by constrained optimisation for the trace inequalities of [1]. Section 7 covers charge-conserving tensor products and spin chains, including momentum sectors, sparse Krylov solvers and particle loss. Section 8 compares with NumPy/SciPy, Section 9 reports performance, and Section 10 discusses limitations.

2 Mathematical setting

Algebra and traces. Let $\mathcal{M} = \bigoplus_{c=1}^C M_{k_c}(\mathbb{C})$ act on $\mathcal{H} = \bigoplus_c \mathbb{C}^{k_c}$; $A \in \mathcal{M}$ is block diagonal, $A = \bigoplus_c A_c$. Every trace on $\mathcal{M}$ is $\text{tr}_w(A) = \sum_c w_c \text{Tr } A_c$ with weights $w_c > 0$. The library implements

$$\text{Tr}_{\text{blunt}} : w_c = 1, \quad \text{Tr}_{\text{norm}} : w_c = \frac{1}{k_c}, \quad \tau_{\text{vN}} : w_c = \frac{1}{Ck_c},$$

where τ_{vN} is the faithful normal tracial state that gives every sector the same weight. For $A \in \mathcal{M}$, $|A| = (A^*A)^{1/2}$; for $A \geq 0$ the Michelson contrast is $\Delta(A) = (\lambda_{\max} - \lambda_{\min})/(\lambda_{\max} + \lambda_{\min}) \in [0, 1]$, with the extreme eigenvalues taken over all sectors; $\Delta(A) = 0$ if and only if A is a multiple of the identity.

States. A state ω on $\mathcal{M}$ is represented by its density $\rho \geq 0$ with respect to Tr_{blunt} , $\omega(A) = \text{Tr}(\rho A)$, $\sum_c \text{Tr } \rho_c = 1$. The restriction of ω to the centre is the classical distribution $p_c = \text{Tr } \rho_c$ over sectors. The density of the same state with respect to tr_w is $\rho_c^{(w)} = \rho_c/w_c$; the library converts between the conventions explicitly, because statements such as “trace preserving” or “the dual map” depend on them as soon as the sectors change.

Maps between sectors. Let $\mathcal{M}_{\text{in}} = \bigoplus_c M_{k_c}$ and $\mathcal{M}_{\text{out}} = \bigoplus_d M_{m_d}$. Every normal completely positive map $\Phi : \mathcal{M}_{\text{in}} \rightarrow \mathcal{M}_{\text{out}}$ (in the Schrödinger picture) is a sum of completely positive maps between blocks,

$$\Phi(\rho)_d = \sum_c \sum_i K_i^{dc} \rho_c (K_i^{dc})^*, \quad K_i^{dc} : \mathbb{C}^{k_c} \rightarrow \mathbb{C}^{m_d}.$$

It preserves Tr_{blunt} iff $\sum_{d,i} (K_i^{dc})^* K_i^{dc} = 1_c$ for all c , and $T_{dc} = \text{Tr} \Phi_{dc}(\rho_c) / \text{Tr} \rho_c$ is the classical transition matrix between sectors. The dual with respect to tr_w on both sides is $\Phi^{*,w}(A)_c = \sum_{d,i} (w_d^{\text{out}} / w_c^{\text{in}}) (K_i^{dc})^* A_d K_i^{dc}$; for sector-preserving maps on one algebra the weights cancel and all three traces give the same dual.

Tensor products and fusion of sectors. $(\bigoplus_c M_{k_c}) \otimes (\bigoplus_d M_{m_d}) = \bigoplus_{(c,d)} M_{k_c m_d}$ is again of Type I with sectors labelled by pairs. If the sectors carry conserved charges q_c, q'_d (particle numbers, magnetisation), the operators of the composite system that conserve only the *total* charge form the larger algebra

$$\mathcal{M}_1 \hat{\otimes} \mathcal{M}_2 = \bigoplus_q M_{n_q}, \quad n_q = \sum_{q_c + q'_d = q} k_c m_d,$$

in which all pairs with the same total charge are merged into one sector. It contains $\mathcal{M}_1 \otimes \mathcal{M}_2$ (block-diagonally in every merged sector), but also operators such as $S_i^+ S_j^-$ that move charge between the factors. For additive charges, the partial trace $(\text{Tr}_2 X)_c = \sum_d \text{Tr}_{m_d} X_{(c,d),(c,d)}$ still lands in $\mathcal{M}_1$: a block between pairs $(c,d) \neq (c',d)$ with equal total charge would need $c = c'$. In both cases the partial trace is a map between sectors in the sense above, and its dual with respect to Tr_{blunt} is the embedding $A \mapsto A \otimes 1$.

Spin chains. For L spins 1/2 with conserved S_{tot}^z , iterating the fused product of single-site algebras $M_1 \oplus M_1$ (charges 0, 1) gives $\mathcal{M} = \bigoplus_{N=0}^L M_{\binom{L}{N}}(\mathbb{C})$, N the number of up spins, which contains every S_{tot}^z -conserving Hamiltonian, e.g. the XXZ chain $H = \sum_{\langle ij \rangle} [\frac{J}{2} (S_i^+ S_j^- + S_i^- S_j^+) + J \Delta S_i^z S_j^z] + \sum_i h_i S_i^z$.

Dynamics. For $H = H^* \in \mathcal{M}$ and jump operators $L_j \in \mathcal{M}$ the Gorini–Kossakowski–Sudarshan–Lindblad equation $\dot{\rho} = -i[H, \rho] + \sum_j \gamma_j (L_j \rho L_j^* - \frac{1}{2} \{L_j^* L_j, \rho\})$ preserves every sector, so the sector probabilities p_c are conserved (superselection).

3 Design and implementation

Software stack. The library is pure Python on top of PyTorch [40] (≥ 1.9 ; tested with 2.x) and NumPy [34]. All dense linear algebra goes through `torch.linalg`, i.e. LAPACK (MKL or OpenBLAS) on CPUs and cuSOLVER, cuBLAS and MAGMA on NVIDIA GPUs; sparse products use PyTorch’s CSR tensors. Automatic differentiation is PyTorch’s reverse mode. The validation and paper scripts additionally use SciPy, pandas and Matplotlib. Packaging follows `pyproject.toml`; the test suite runs in continuous integration on Python 3.10 and 3.12 (CPU), and the paper scripts record the PyTorch, CUDA and cuDNN versions, the device and its compute capability in every generated table.

Workflow. A computation proceeds in four steps. (1) An algebra is declared by its sector sizes, optionally sector charges, number field, precision and device (`TypeIAlgebra`, or a model such as `SpinChain` that creates it). (2) Objects are created in it: random operators with prescribed spectra, states, channels, Hamiltonians, or differentiable parametrisations. (3) They are combined lazily (products, functional calculus, channel application, tensor products, time evolution). (4) Functionals

are evaluated for the whole batch at once (traces, spectra, contrasts, entropies, sector probabilities) and reduced to statistics, or fed into an optimiser that updates the parameters of step (2). Every step keeps the batch dimension, so the same code serves one operator and 10^5 samples.

challenge	solution in the library
(i) blocks of different size on batched kernels	padded $(B, C, k_{\max}, k_{\max})$ layout, one batched call per sector; single-sector algebras and sparse sector Hamiltonians where padding would dominate
(ii) conventions between sectors	states stored as Tr_{blunt} -densities with explicit conversions; duals of maps between sectors computed with the weights of the chosen trace
(iii) exact random ensembles	Mezzadri's QR algorithm with phase correction, $\det^{-1/n}$ for $SU(n)$, $W^T W$ and $W^R W$ for COE/CSE, validated against Weingarten moments and spacing statistics
(iv) singular gradients at degenerate spectra	parametrisations that build operators from their spectra (no eigendecomposition in the forward pass), exact $ X $ from the parametrisation
(v) extremes missed by sampling	batched multistart optimisation with exact contrast constraints
(vi) time and memory budgets	probing of batched kernels with an ETA warning, memory estimates checked against the free device memory, ETA for time stepping
(vii) field and precision	two switches per algebra, conversions between them, measured trade-offs

Table 2: Implementation challenges (Section 1) and how they are addressed.

Data layout. An operator is a tensor of shape $(B, C, k_{\max}, k_{\max})$: B samples, C sectors, block c in the top-left $k_c \times k_c$ corner and zeros elsewhere. Linear algebra is applied to the batch of active blocks of each sector, so one call to a batched LAPACK/cuSOLVER routine serves all samples. The padding costs memory when the sector sizes differ strongly (for a chain of 14 spins, 77 % of a padded operator is zeros); the library warns in that case, and **SpinChain** lets one work in single sectors $M_{(N)}^L$ without padding, which is what dense diagonalisation needs anyway.

Objects. **Operator** (with **DensityMatrix** as a subclass for states), **Channel** (sector preserving) and **InterSectorChannel** (between sectors or algebras) are thin wrappers around such tensors; Table 3 lists the modules. Operations on operators are lazy: they record how to compute the result and do so on first use; the result is cached and the recipe dropped, so intermediates of long expressions can be freed. Laziness defers and skips work but does not fuse operations; where this matters the library uses direct formulas, e.g. $\text{Tr}(\rho A) = \sum_{ij} \rho_{ij} A_{ji}$ in $O(k^2)$ instead of forming ρA .

module	content
algebra	TypeIAlgebra , Operator : random operators, unitary ensembles, traces, functional calculus
states	DensityMatrix , random states, Gibbs states, Lüders rule, entropies, fidelity
channels	Channel , InterSectorChannel , Choi matrices, standard and random channels
composite	plain and fused (charge-conserving) tensor products, kron , partial traces
chains	SpinChain : $U(1)$ and momentum sectors, XXZ Hamiltonians with batched disorder, sparse form, reduced states, jump operators between sectors
krylov	sparse sector Hamiltonians, batched Lanczos ground states, Krylov $e^{-itH}\psi$
dynamics	exact Schrödinger/von Neumann evolution, RK4, Lindblad equation, $e^{t\mathcal{L}}$
optimize	parametrisations with prescribed contrast, batched multistart extremize
cost	run-time and memory estimates, warnings and errors

Table 3: Modules of **torch_vn_algebra**.

Random unitaries. Haar unitaries on $U(n)$ and $O(n)$ are obtained from the QR decomposition of a Ginibre matrix with i.i.d. circular Gaussian entries after fixing the phases of $\text{diag } R$ [11]; $SU(n)$ and $SO(n)$ by multiplying with $\det^{-1/n}$ or by a sign flip of one column; the circular orthogonal and symplectic ensembles as $W^T W$ and $W^R W$ with W Haar on $U(n)$ [12]. Section 5 checks all of them.

Differentiation. All operations are composed of differentiable PyTorch primitives, so gradients with respect to the matrix entries are available; eigenvalue and singular-value decompositions have singular gradients at degenerate spectra, which the parametrisations of Section 6 avoid.

GPUs, tensor cores and TF32. The library computes in FP32 or FP64 (real or complex); it does not use half precision. GPU generations differ strongly in what this costs. Tensor cores, absent on the P100 used for the first version, accelerate matrix products on Volta and later GPUs; from Ampere (A100) on, FP32 products may be executed in the TF32 format with a 10-bit mantissa, which is fast but accurate only to about 10^{-3} relative. PyTorch keeps true FP32 for matrix products by default, and so does the library; the paper scripts expose a `-tf32` switch to measure the effect and record the setting. Eigen- and singular-value solvers do not use tensor cores. FP64 throughput is half of FP32 on data-centre GPUs (P100, V100, A100, H100), the latter two with FP64 tensor cores, but only 1/32–1/64 on consumer GPUs. We therefore recommend a data-centre GPU for double-precision work; the full-size runs of this paper are intended for an A100 or newer, and all generated tables state the hardware they were produced on.

Number field and precision

Every algebra is created with two independent switches, `complex_valued` (real or complex matrices) and `precision` (`'single'` or `'double'`), giving the storage types `float32`, `float64`, `complex64` and `complex128`; everything built on the algebra (random operators, states, channels, chains, parametrisations) inherits them. `alg.like(complex_valued=..., precision=...)` creates the same sector structure with other numerics and `op.cast(alg2)` moves an operator there, e.g. to explore in single precision and re-evaluate the interesting cases in double precision.

The field is part of the model. Choosing real matrices is not only a matter of speed: the symmetry group and the random ensembles change. Haar matrices of a real algebra are orthogonal, so moments, level statistics and extremal values refer to $O(n)$ instead of $U(n)$ (e.g. $\mathbb{E}|U_{11}|^4 = 3/(n(n+2))$ instead of $2/(n(n+1))$, Table 6); random real symmetric and complex Hermitian matrices have different level statistics, with mean spacing ratios 0.5321 and 0.6021 for GOE and GUE [17] (computed with the library). Real arithmetic is *exact* (loses nothing) when the problem is real: spectra and eigenvectors of real symmetric matrices, ground states and their entanglement for Hamiltonians without complex hoppings, statistics of time-reversal-invariant systems, and also some extremal problems — for real $X, Y \geq 0$ the supremum of $|\text{Tr}(XUY)|$ over unitaries is attained by a real orthogonal matrix (the polar decomposition of the real matrix YX is real). Complex matrices are *required* for unitary time evolution e^{-iHt} and Lindblad dynamics with a Hamiltonian (the library refuses to run them in a real algebra), for Haar unitaries, CUE/CSE and $SU(n)$, for random pure states and quantities such as Page’s entropy, and for systems without time-reversal symmetry (fluxes, complex couplings).

Precision. Single precision resolves about 10^{-7} relative to the largest entry. It is sufficient for Monte Carlo statistics whose statistical error is much larger (moments, histograms, spacing ratios with 10^3 – 10^5 samples) and for the first, exploratory pass of an optimisation. Double precision is needed when the quantity of interest is small compared with the operators: an inequality checked

near equality, small spectral gaps, entropies and logarithms of nearly singular states, contrasts close to 1, comparisons with exact results beyond six digits, long RK4 integrations and optimisation to high relative accuracy. Table 5 shows how drastic the difference can be: the von Neumann entropy of an almost pure state with eigenvalues of order 10^{-9} is wrong by more than 100 % in single precision, and the round-off floor of a trace inequality that holds with equality is 10^{-5} in single and 10^{-13} in double precision.

Cost. An entry takes 4, 8, 8 and 16 bytes for `float32`, `float64`, `complex64` and `complex128`, so memory, and the batch that fits, changes by a factor up to four (`alg.bytes_per_operator(B)` reports it). A complex multiply–add costs about four real ones, and complex eigen- and singular-value solvers are two to four times slower than real ones. Double precision costs about a factor two on CPUs, but on GPUs it depends on the hardware: data-centre GPUs execute FP64 at half the FP32 rate, consumer GPUs at 1/32–1/64 of it, so that double precision there can be an order of magnitude slower. Table 4 gives the measured ratios for the current run; the recommendation that follows is to use the smallest type that is exact for the model (real if the problem is real) and the precision that the size of the effect requires.

type	B/entry	MB/batch	generate	eigh f(A)	SVD A	channel	expm	opt. step
<code>float32</code>	4	1	1	1	1	1	1	1
<code>float64</code>	8	2	1.5	1.5	1.1	1.7	1.7	1.2
<code>complex64</code>	8	2	1.7	1.4	1.5	1.9	2	1.2
<code>complex128</code>	16	4	2.2	1.5	1.7	2.4	2	2.2

Table 4: Memory and time relative to `float32` for $C = 8$ sectors of size $k = 16$ and a batch of 128 operators (quick run on CPU (4 threads); `float32` times per sample: generate: $105\ \mu\text{s}$, `eigh f(A)`: $167\ \mu\text{s}$, `SVD |A|`: $298\ \mu\text{s}$, `channel`: $30.7\ \mu\text{s}$, `expm`: $57.4\ \mu\text{s}$, `opt. step`: $780\ \mu\text{s}$).

type	$\max \ UU^* - 1\ $	$\ \sqrt{A}^2 - A\ /\ A\ $	rel. error $S(\rho)$, $\rho \approx \text{pure}$	$ z $ at equality
<code>float32</code>	8.3×10^{-7}	1.9×10^{-6}	2.3×10^0	2.3×10^{-5}
<code>float64</code>	1.4×10^{-15}	4.0×10^{-15}	5.1×10^{-9}	5.7×10^{-14}
<code>complex64</code>	8.3×10^{-7}	2.2×10^{-6}	2.0×10^0	2.7×10^{-5}
<code>complex128</code>	1.1×10^{-15}	4.4×10^{-15}	5.0×10^{-9}	5.0×10^{-14}

Table 5: Accuracy of the same computations in the four storage types: Haar unitaries ($n = 64$), SVD square root, entropy of $\rho = (1 - \varepsilon)|0\rangle\langle 0| + \varepsilon 1/n$ with $\varepsilon = 10^{-7}$, and the round-off floor of $z = \text{Tr}(Y|X|Y) - \text{Tr}|YXY|$ for commuting X, Y (exactly 0).

Safeguards. Batched computations make it easy to request hours of work or more memory than the device has. The `cost` module therefore (a) times the first few items of every large batched eigen-, singular-value, QR or exponential decomposition and issues a warning with the expected total time when it exceeds a threshold (10 s by default), (b) compares the estimated peak memory of large allocations with the free memory of the device, warning above 50 % and raising an `InsufficientMemoryError` above 90 %, and (c) reports the expected duration of time-stepping and optimisation loops after their second step, optionally with a progress bar. All thresholds can be changed, and the checks can be switched off locally.

4 Usage by example

The listings of this section are complete programs from `examples/paper`; the test suite runs them, so they stay in sync with the code. They run on a CPU in seconds and use a GPU when available.

Random operators with a prescribed contrast. Listing 1 builds 10^4 positive operators in $M_2 \oplus M_3 \oplus M_4$ whose spectra have contrast 0.6 and whose eigenvectors are Haar-random in every sector, and evaluates traces and spectral functions.

Listing 1: Random operators (`ex1_random_operators.py`).

```
"""Random operators with a prescribed spectrum in M = M_2 (+) M_3 (+) M_4."""
import torch
from torch_vn_algebra import TypeIAlgebra

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
alg = TypeIAlgebra(n_factors=[2, 3, 4], k_factors=[2, 3, 4], device=dev)
B = 10_000 # Monte Carlo samples, processed as one batch

def contrast_sampler(delta):
    """Spectra in [lo, 1] with lambda_max = 1, lambda_min = lo, i.e. Michelson contrast delta."""
    lo = (1 - delta) / (1 + delta)

    def sampler(dim):
        lam = lo + (1 - lo) * torch.rand(B, dim, device=dev)
        lam[:, 0], lam[:, -1] = 1.0, lo
        return lam
    return sampler

# A_c = U_c diag(lambda) U_c^*, with independent Haar unitaries U_c in every block and sample
X = alg.operator_from_eigenvalues(contrast_sampler(0.6), batch_size=B,
                                force_positive=True, force_self_adjoint=True)
print(X.matrix.shape) # (10000, 3, 4, 4): batch, sector, padded block
print(X.michelson_contrast[:4]) # 0.6 for every sample
print(X.Tr_blunt().real.mean(), X.Tr_norm().real.mean(), X.tau_vN().real.mean())

# functional calculus through the spectral theorem, block by block
R = X.sqrt()
print((R @ R - X).frobenius_norm().max()) # ~1e-6 in single precision
print(X.apply_function(torch.log).tau_vN().real.mean()) # tau_vN(log X)
```

States, measurement and trace conventions. Listing 2 conditions random states on the outcome of a projective measurement (Lüders rule), which moves all probability into one sector, and checks that expectation values do not depend on the trace used to represent the state.

Listing 2: States and conditioning (`ex2_states_measurement.py`).

```
"""States on M, sector probabilities, Lueders conditioning and trace conventions."""
import torch
from torch_vn_algebra import TypeIAlgebra, DensityMatrix, states

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
alg = TypeIAlgebra([2, 3], [2, 3], device=dev) # two superselection sectors

rho = states.random_density_matrix(alg, batch_size=4) # Hilbert-Schmidt random states
```

```

print(rho.sector_probabilities())                                # p_c = Tr rho_c, rows sum to 1

# measure the projection onto the first basis vector of sector 0
P = alg.from_blocks([torch.diag(torch.tensor([1.0, 0.0])), torch.zeros(3, 3)])
post, prob = rho.condition_on(P)                                # rho|P = P rho P / Tr(rho P)
print(prob, post.sector_probabilities())                        # all weight now in sector 0

# the same state as a density w.r.t. tau_vN: omega(A) = tau_vN(d A) = Tr(rho A)
A = alg.operator_from_eigenvalues(lambda d: torch.randn(4, d), batch_size=4,
                                   force_self_adjoint=True)
d_tau = rho.density('tau_vN')
print(rho.expectation(A).real, (alg.operator(d_tau) @ A).tau_vN().real)
back = DensityMatrix.from_density(alg, d_tau, trace='tau_vN')
print((back.matrix - rho.matrix).abs().max())

```

A map between sectors. Two fermionic modes have sectors $N = 0, 1, 2$ of dimensions 1, 2, 1. Particle loss with Kraus operators $\sqrt{1-g}N$, $\sqrt{g}a_1$, $\sqrt{g}a_2$ maps sector N to N and $N-1$. Listing 3 builds it from its blocks; the classical part is the transition matrix of a death process,

$$T = \begin{pmatrix} 1 & g & 0 \\ 0 & 1-g & 2g \\ 0 & 0 & 1-2g \end{pmatrix},$$

which the program prints. The last lines illustrate a simple fact: a unital positive map cannot increase the Michelson contrast, since $\lambda_{\min}1 \leq A \leq \lambda_{\max}1$ implies the same bounds for $\Phi(A)$; the non-unital loss channel increases it for most samples.

Listing 3: Particle loss between sectors (`ex3_channels.py`).

```

"""Particle loss between charge sectors of two fermionic modes, as an InterSectorChannel."""
import torch
from torch_vn_algebra import TypeIAlgebra, InterSectorChannel, channels, states

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
# sectors N = 0, 1, 2 with bases |00>, |10>, |01>, |11>
alg = TypeIAlgebra([1, 2, 1], [1, 2, 1], device=dev)
g = 0.2                                # loss probability per particle
s, z = g ** 0.5, torch.zeros
# Kraus operators K_0 = sqrt(1 - g)N, K_1 = sqrt(g) a_1, K_2 = sqrt(g) a_2, split into blocks (d, c)
blocks = {
    (0, 0): [torch.ones(1, 1)],
    (1, 1): [(1 - g) ** 0.5 * torch.eye(2)],
    (2, 2): [(1 - 2 * g) ** 0.5 * torch.ones(1, 1)],
    (0, 1): [z(1, 2), s * torch.tensor([[1.0, 0.0]]), s * torch.tensor([[0.0, 1.0]])],
    (1, 2): [z(2, 1), s * torch.tensor([[0.0], [1.0]]), s * torch.tensor([[1.0], [0.0]])],
}
loss = InterSectorChannel.from_blocks(alg, alg, blocks)
print(loss.is_trace_preserving())
print(loss.transition_matrix()[0])      # classical death process on N

rho = states.random_density_matrix(alg, batch_size=3)
print(loss(rho).sector_probabilities()) # still a DensityMatrix

# unital channels cannot increase the Michelson contrast, non-unital ones can
X = alg.operator_from_eigenvalues(lambda d: 0.5 + torch.rand(1000, d, device=dev),
                                   batch_size=1000, force_positive=True)
mix = channels.random_mixed_unitary_channel(alg, 3, batch_size=1000)

```

```
print(bool((mix(X).michelson_contrast <= X.michelson_contrast + 1e-6).all()))
print((loss(X).michelson_contrast > X.michelson_contrast).float().mean())
```

Open-system dynamics. Listing 4 integrates spontaneous emission of a two-level atom with the RK4 solver and with the exact exponential of the Lindblad generator, and compares both with $P_e(t) = P_e(0)e^{-\gamma t}$ and $|\rho_{eg}(t)| = |\rho_{eg}(0)|e^{-\gamma t/2}$.

Listing 4: Spontaneous emission (ex4_dynamics.py).

```
"""Spontaneous emission of a two-level atom: Lindblad evolution against the exact solution."""
import math
import torch
from torch_vn_algebra import TypeIAlgebra, DensityMatrix, dynamics

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
alg = TypeIAlgebra([2], [2], device=dev, precision='double')
omega, gamma = 1.0, 0.3
H = alg.from_blocks([torch.diag(torch.tensor([0.5 * omega, -0.5 * omega]))]) # basis |e>, |g>
L = alg.from_blocks([torch.tensor([0.0, 0.0], [1.0, 0.0])]) # sigma_- = |g><e|
psi = torch.tensor([0.6, 0.8], dtype=torch.complex128)
rho0 = DensityMatrix(alg, matrix=torch.outer(psi, psi.conj())[None, None].to(dev))

times = [0.0, 1.0, 2.0, 5.0]
rk4 = dynamics.lindblad_evolve(rho0, H, [L], times, rates=[gamma], substeps=100)
for t, rho in zip(times, rk4):
    exact = dynamics.lindblad_channel(alg, H, [L], t=t, rates=[gamma])(rho0) # exp(tL)
    p_e = 0.36 * math.exp(-gamma * t) # analytic excited population
    coh = 0.48 * math.exp(-gamma * t / 2) # analytic |rho_eg|
    m = rho.matrix[0, 0]
    print(f"t={t}: P_e {m[0, 0].real:.6f} (exact {p_e:.6f}), |rho_eg| {m[0, 1].abs():.6f} "
          f"(exact {coh:.6f}), RK4 vs exp(tL): {(rho.matrix - exact.matrix).abs().max():.1e}")
```

Tensor products. Listing 5 estimates the mean entanglement entropy of random pure states on $\mathbb{C}^m \otimes \mathbb{C}^n$ and compares it with Page’s formula [13].

Listing 5: Page entropy (ex5_page_entropy.py).

```
"""Average entanglement entropy of random pure states on C^m (x) C^n (Page, 1993)."""
import torch
from torch_vn_algebra import TypeIAlgebra, tensor_product, partial_trace, states

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
m, n, B = 4, 8, 20_000
A = TypeIAlgebra([m], [m], device=dev, precision='double')
Bsys = TypeIAlgebra([n], [n], device=dev, precision='double')
AB = tensor_product(A, Bsys)

psi = states.random_density_matrix(AB, batch_size=B, rank=1) # Haar-random pure states
S = states.von_neumann_entropy(partial_trace(psi, keep=1)) # entropy of rho_A

page = sum(1.0 / k for k in range(n + 1, m * n + 1)) - (m - 1) / (2 * n)
print(f"<S_A> = {S.mean():.5f} +- {S.std() / B ** 0.5:.5f}, Page: {page:.5f}")
```

Searching for a violation. Listing 6 compares Haar sampling with batched gradient ascent for $\sup_U (|\text{Tr}(XUY)| - \text{Tr}(XY))$, whose exact value $\|YX\|_1 - \text{Tr}(XY)$ is positive for non-commuting $X, Y \geq 0$. Section 6 studies this systematically.

Listing 6: Sampling versus optimisation (ex6_inequality_search.py).

```

"""Searching for violations of  $|\text{Tr}(XUY)| \leq \text{Tr}(XY)$ : random sampling versus gradient ascent."""
import torch
from torch_vn_algebra import TypeIAlgebra

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
torch.manual_seed(0)
k = 16
alg = TypeIAlgebra([k], [k], device=dev, precision='double')
pos = lambda d: 0.2 + torch.rand(1, d)
X = alg.operator_from_eigenvalues(pos, force_positive=True, force_self_adjoint=True)
Y = alg.operator_from_eigenvalues(pos, force_positive=True, force_self_adjoint=True)
trXY = (X @ Y).trace.real

def z(U):
    #  $z(U) = |\text{Tr}(XUY)| - \text{Tr}(XY)$ , batched over U
    return (X @ U @ Y).trace.abs() - trXY

# 1) Monte Carlo over Haar unitaries
print("sampling,  $10^5$  Haar U:", z(alg.random_unitary_operator(100_000)).max().item())

# 2) gradient ascent over  $U = U_0 \exp(G - G^*)$ , 64 random starts optimised in parallel;
# the exponential chart is re-centred at the current point every 50 steps
U0 = alg.random_unitary_operator(64).matrix
for _ in range(10):
    G = torch.zeros(64, 1, k, k, dtype=alg.hilbert.dtype, device=dev, requires_grad=True)
    opt = torch.optim.Adam([G], lr=0.05)
    for step in range(50):
        U = alg.operator(U0) @ alg.operator(G - G.conj().transpose(-2, -1)).expm()
        loss = -z(U).sum()
        opt.zero_grad()
        loss.backward()
        opt.step()
    U0 = U.matrix.detach()
print("gradient ascent:      ", z(alg.operator(U0)).max().item())

# exact supremum:  $\sup_U |\text{Tr}(U YX)| = \|YX\|_1$ 
print("exact sup:          ", ((Y @ X).trace_norm() - trXY).item())

```

Spin chains. Listing 7 works with a Heisenberg ring of ten spins. A uniform field moves the ground state through the sectors until the chain is fully polarised above the saturation field $h_s = 2J$; 100 realisations of random fields are diagonalised in one batch to compute the mean level-spacing ratio; and the entanglement entropy of the ground state is computed from reduced states. Section 7 checks these tools against exact results.

Listing 7: Spin chains (ex7_spin_chain.py).

```

"""Spin chains with conserved  $S^z$ : sectors, disorder averages and entanglement."""
import torch
from torch_vn_algebra import SpinChain, DensityMatrix

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
chain = SpinChain(L=10, boundary='periodic', device=dev) # sectors N = 0..10, dims binom(10, N)
print(chain.algebra.k_factors)

# Heisenberg ring in a uniform field: the ground state changes sector as h grows

```

```

for h in (0.0, 1.0, 2.5):
    w = [blk[0] for blk, _ in chain.heisenberg(h=h).eigh()]
    print(f"h={h}: ground state in sector N={min(range(11), key=lambda N: w[N].min())}")

# random fields h_i ~ U[-W, W]: 100 disorder realisations in one batch, sector N = 5 only
for W in (0.5, 8.0):
    H = chain.random_field_heisenberg(W, batch_size=100, sector=5)
    E, _ = H.eigh()[0] # (100, 252)
    r = SpinChain.level_spacing_ratio(E[:, 63:189]).mean()
    print(f"W={W}: <r> = {r:.3f} (GOE 0.531, Poisson 0.386)")

# entanglement entropy of the ground state of the clean chain, left block of n sites
E, V = chain.heisenberg(sector=5).eigh()[0]
psi = V[0, :, 0]
rho = DensityMatrix(chain.sector_algebra(5), matrix=torch.outer(psi, psi.conj())[None, None])
print([round(chain.entanglement_entropy(rho, n, sector=5).item(), 4) for n in range(1, 6)])

```

Large sectors and open chains. Dense matrices of a sector of dimension $\binom{L}{L/2}$ stop fitting at about $L = 16$ – 18 . Listing 8 uses the sparse form of the Hamiltonian (a batch of diagonals and one shared hopping matrix) with Lanczos and Krylov solvers, splits a ring into momentum sectors, and lets particles leak out of the chain through Lindblad jumps between sectors.

Listing 8: Large sectors and open chains (ex8_large_and_open_chains.py).

```

"""Beyond dense matrices: Lanczos, Krylov evolution, momentum sectors and particle loss."""
import math
import torch
from torch_vn_algebra import SpinChain, krylov, dynamics

dev = 'cuda' if torch.cuda.is_available() else 'cpu'

# Lanczos in a sector of dimension binom(16, 8) = 12870, sparse Hamiltonian
ring = SpinChain(16, 'periodic', complex_valued=False, device=dev)
E0, psi, res = krylov.ground_state(ring.xx_sparse(sector=8))
print(f"Heisenberg ring L=16: E0 = {E0.item():.9f} (exact -7.142296361), residual {res.item():.1e}")

# Krylov evolution of a domain wall in the XX chain, L = 14 (sector dimension 3432)
chain = SpinChain(14, device=dev)
psi0 = chain.vector_in_sector('1' * 7 + '0' * 7)
psi_t = krylov.evolve(chain.xx_sparse(J=1.0, Delta=0.0, sector=7), psi0[None], [0.0, 3.0])
mag = (psi_t[-1, 0].abs() ** 2) @ chain.bits(7).to(torch.float64) - 0.5
print("S^z_i(t=3):", [round(x, 3) for x in mag.tolist()])

# translation symmetry: blocks of fixed momentum k = 2 pi m / L inside the sector N = 5 of a ring of
# 10
ring10 = SpinChain(10, 'periodic', device=dev)
Hk = ring10.xx_momentum(J=1.0, Delta=1.0, sector=5)
print("momentum block sizes:", ring10.momentum_dims(5))
e0 = {m: w[0].min().item() for m, (w, _) in zip(Hk.algebra.charges, Hk.eigh())}
print("ground state momentum m =", min(e0, key=e0.get), "(pi for L/2 odd)")

# particle loss S_-i on every site with rate g: the sector distribution stays binomial
small = SpinChain(5, 'periodic', device=dev)
rho = dynamics.lindblad_evolve(small.basis_state('11011'), small.xx(1.0, 0.7),
                               [small.lowering(i) for i in range(5)], [0.0, 1.0], rates=[0.4] * 5)

[-1]
q = math.exp(-0.4)
print("p_N(t=1):", [round(x, 6) for x in rho.sector_probabilities()[0].tolist()])

```

```
print("binomial:", [round(math.comb(4, n) * q ** n * (1 - q) ** (4 - n), 6) for n in range(5)] +
      [0.0])
```

A noisy quantum circuit with a conserved quantity. The quantum alternating operator ansatz solves constrained optimisation problems with “XY mixers” $\sum (X_i X_j + Y_i Y_j)$ that keep the Hamming weight of the computational basis states, so the evolution stays in the feasible subspace [29]. In the language of this paper the qubit register carries the algebra $\bigoplus_{N=0}^n M_{\binom{n}{N}}$ of Hamming-weight sectors, the mixer is an element of it, amplitude damping (T_1) is a Lindblad jump between sectors and pure dephasing (T_2) a jump inside them, and reading out the Hamming weight is the measurement of a central projection. Listing 9 evolves $|111000\rangle$ under the mixer with both kinds of noise and post-selects on the initial weight. With T_1 noise only, the post-selected fidelity is exactly one and the acceptance probability is $e^{-w\gamma_1 T}$: the no-jump evolution is generated by $H - \frac{i}{2}\gamma_1 N$, and N is central, so it only rescales the state. This exact statement is part of the test suite. Dephasing stays inside the sector and is not detected:

γ_1	γ_2	$P(\text{weight } 3)$	fidelity	after post-selection
0.02	0	0.8869	0.8869	1.0000
0	0.02	1.0000	0.9730	0.9730
0.02	0.02	0.8869	0.8629	0.9730
0.1	0.1	0.5488	0.4805	0.8755

Listing 9: XY mixer with T_1/T_2 noise and post-selection (ex9_qaoa_xy_mixer.py).

```
"""
Quantum-computing example: an XY mixer (QAOA for constrained problems) on 6 qubits keeps the
Hamming weight, i.e. it acts inside one sector of  $M = (+)_N M_{\text{binom}(6,N)}$ . Amplitude damping (T1)
moves weight to lower sectors, dephasing (T2) does not; measuring the Hamming weight - a central
projection of  $M$  - detects the first kind of error and post-selection restores part of the fidelity.
"""

import torch
from torch_vn_algebra import SpinChain, dynamics, states

dev = 'cuda' if torch.cuda.is_available() else 'cpu'
n, w, T = 6, 3, 2.0 # qubits, Hamming weight, evolution time
qubits = SpinChain(n, 'periodic', device=dev) # ring of qubits, sectors = Hamming
weights
H = qubits.xx(J=1.0, Delta=0.0) # XY mixer sum (X_i X_j + Y_i Y_j) / 4
rho0 = qubits.basis_state('111000') # feasible initial state, weight 3
ideal = dynamics.von_neumann(rho0, H, [T])[0] # noiseless reference (pure)
P = qubits.algebra.central([1.0 if N == w else 0.0 for N in range(n + 1)]) # Hamming-weight
projector

T1_jumps = [qubits.lowering(i) for i in range(n)] # |1> -> |0>: between sectors
T2_jumps = [qubits.sz(i) for i in range(n)] # dephasing: inside sectors
print(" gamma1 gamma2 P(weight 3) fidelity fidelity after post-selection")
for g1, g2 in ((0.0, 0.0), (0.02, 0.0), (0.0, 0.02), (0.02, 0.02), (0.1, 0.1)):
    rho = dynamics.lindblad_evolve(rho0, H, T1_jumps + T2_jumps, [0.0, T],
                                   rates=[g1] * n + [g2] * n, substeps=80)[-1]
    post, p_ok = rho.condition_on(P) # Lueders rule for the central
    projection
    F = states.trace_of_product(rho, ideal).real.item() # <psi_ideal| rho |psi_ideal>
    F_post = states.trace_of_product(post, ideal).real.item()
    print(f" {g1:5.2f} {g2:5.2f} {p_ok.item():10.4f} {F:8.4f} {F_post:8.4f}")
```

5 Validation against exact results

Table 6 compares Monte Carlo estimates produced with the library (double precision) with exact values: fourth moments of Haar unitaries and orthogonal matrices; the Weingarten-calculus value $\mathbb{E} \tau((AUBU^*)^2) = -1/(N^2 - 1)$ for traceless involutions $A = B$ and Haar U ; Page's formula for $\langle S(\rho_A) \rangle$; and the mean purity $(k + r)/(kr + 1)$ of induced random states of rank r [14]. The last column is the deviation in units of the standard error; all are of order one. For the circular ensembles the Wigner surmise is only an approximation to the exact spacing distribution, so no z -score is given; Figure 1 shows the distributions, with L^1 distances 0.031, 0.021 and 0.029 for $\beta = 1, 2, 4$ from the surmise; level repulsion $P(s) \sim s^\beta$ is clearly visible. The spontaneous-emission example agrees with the analytic solution to 2.2×10^{-16} (exact exponential) and 2.1×10^{-8} (RK4).

quantity	parameters	exact	Monte Carlo	std. error	z
$\mathbb{E} U_{11} ^4, U(n)$	$n = 2$	0.333333	0.335015	1.5×10^{-3}	+1.1
$\mathbb{E} U_{11} ^2, U(n)$	$n = 2$	0.5	0.5014	1.4×10^{-3}	+1.0
$\mathbb{E} U_{11} ^4, U(n)$	$n = 4$	0.1	0.0999686	6.8×10^{-4}	-0.0
$\mathbb{E} U_{11} ^4, U(n)$	$n = 8$	0.0277778	0.0277718	2.4×10^{-4}	-0.0
$\mathbb{E} U_{11} ^4, U(n)$	$n = 16$	0.00735294	0.00737778	7.1×10^{-5}	+0.4
$\mathbb{E} U_{11} ^4, U(n)$	$n = 32$	0.00189394	0.00188255	2.0×10^{-5}	-0.6
$\mathbb{E} U_{11} ^2, U(n)$	$n = 32$	0.03125	0.031172	1.5×10^{-4}	-0.5
$\mathbb{E} U_{11} ^4, O(n)$	$n = 2$	0.375	0.373641	1.8×10^{-3}	-0.7
$\mathbb{E} U_{11} ^2, O(n)$	$n = 2$	0.5	0.498956	1.8×10^{-3}	-0.6
$\mathbb{E} U_{11} ^4, O(n)$	$n = 4$	0.125	0.12365	9.8×10^{-4}	-1.4
$\mathbb{E} U_{11} ^4, O(n)$	$n = 8$	0.0375	0.0378452	4.0×10^{-4}	+0.9
$\mathbb{E} U_{11} ^4, O(n)$	$n = 16$	0.0104167	0.0104973	1.4×10^{-4}	+0.6
$\mathbb{E} U_{11} ^4, O(n)$	$n = 32$	0.00275735	0.0027189	3.9×10^{-5}	-1.0
$\mathbb{E} U_{11} ^2, O(n)$	$n = 32$	0.03125	0.0311461	2.1×10^{-4}	-0.5
$\mathbb{E} \tau((AUBU^*)^2)$	$N = 4$	-0.0666667	-0.0668132	1.8×10^{-3}	-0.1
$\mathbb{E} \tau((AUBU^*)^2)$	$N = 16$	-0.00392157	-0.00342506	4.0×10^{-4}	+1.3
$\mathbb{E} \tau((AUBU^*)^2)$	$N = 64$	-0.0002442	-0.000192229	2.0×10^{-4}	+0.3
$\langle s^2 \rangle$, COE	$N = 32$	1.27324	1.28476	5.2×10^{-3}	—
$\langle s^2 \rangle$, CUE	$N = 32$	1.1781	1.17783	3.8×10^{-3}	—
$\langle s^2 \rangle$, CSE	$N = 32$	1.10447	1.10321	3.9×10^{-3}	—
$\langle S(\rho_A) \rangle$	$m = 2, n = 8$	0.600372	0.600235	9.6×10^{-4}	-0.1
$\langle S(\rho_A) \rangle$	$m = 4, n = 8$	1.15314	1.15389	1.0×10^{-3}	+0.7
$\langle S(\rho_A) \rangle$	$m = 8, n = 8$	1.58853	1.58838	8.7×10^{-4}	-0.2
$\mathbb{E} \text{Tr} \rho^2$	$k = 4, r = 2$	0.666667	0.666228	1.0×10^{-3}	-0.4
$\mathbb{E} \text{Tr} \rho^2$	$k = 4, r = 4$	0.470588	0.470432	6.8×10^{-4}	-0.2
$\mathbb{E} \text{Tr} \rho^2$	$k = 16, r = 4$	0.307692	0.307736	2.0×10^{-4}	+0.2
$\mathbb{E} \text{Tr} \rho^2$	$k = 16, r = 16$	0.124514	0.124468	5.4×10^{-5}	-0.8

Table 6: Monte Carlo estimates versus exact values (quick run on CPU (4 threads), PyTorch 2.14.1+cu130).

Trace functionals. The three traces are checked on random non-Hermitian operators in $M_2 \oplus M_3 \oplus M_5$ (Table 7): normalisation on the identity ($\sum_c k_c, C$ and 1), the tracial property $\text{tr}(AB) = \text{tr}(BA)$, invariance under unitaries of $\mathcal{M}$, and positivity $\text{tr}(A^*A) > 0$ (faithfulness); the last column is the smallest ratio $\text{tr}(A^*A)/\|A\|_F^2$ over the samples, which is bounded below by $\min_c w_c$. All three return values in the number field of the algebra (complex traces of non-Hermitian operators).

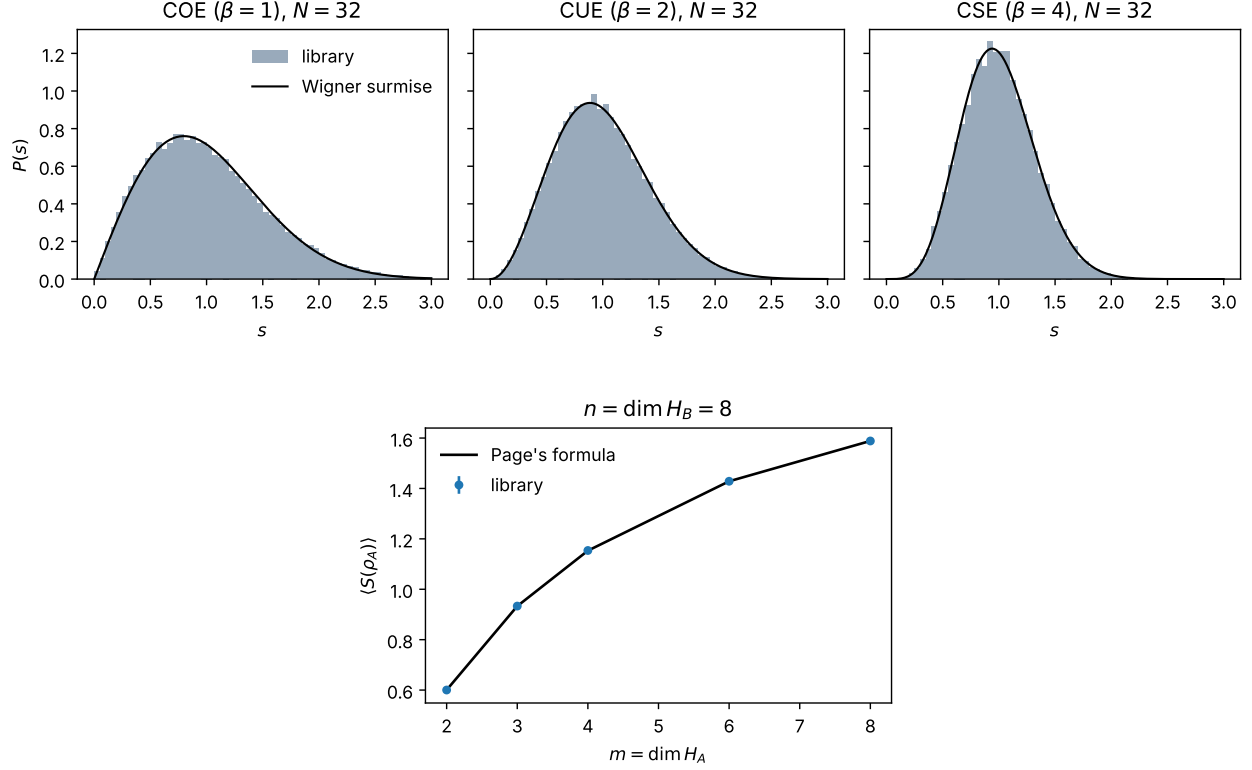


Figure 1: Top: nearest-neighbour spacings of COE, CUE and CSE eigenphases generated by the library, against the Wigner surmise. Bottom: mean entanglement entropy of random pure states against Page’s formula (error bars: two standard errors).

trace	error of $\text{tr}(1)$	$\max \text{tr}(AB) - \text{tr}(BA) $	$\max \text{tr}(UAU^*) - \text{tr}(A) $	$\min \text{tr}(A^*A)/\ A\ _F^2$
Tr_{blunt}	0	4.0×10^{-15}	7.1×10^{-15}	1.000
Tr_{norm}	0	8.9×10^{-16}	1.6×10^{-15}	0.223
τ_{vN}	0	4.4×10^{-16}	5.2×10^{-16}	0.074

Table 7: Identities of the three trace functionals, 2000 random operators in double precision.

6 From sampling to optimisation

The first version of this work [1] studied three trace functionals by Monte Carlo:

$$\begin{aligned}
\text{Exp. 1: } z &= |\text{Tr}(XUY)| - \text{Tr}(XY), & X, Y \geq 0, U \in \mathcal{M} \text{ unitary,} \\
\text{Exp. 2: } z &= \text{Tr}|XY| - \text{Tr}(|X||Y|), & X = UX_0, Y = VY_0, X_0, Y_0 \geq 0, \\
\text{Exp. 3: } z &= \text{Tr}(Y|X|Y) - \text{Tr}|YXY|, & X = X^*, Y \geq 0,
\end{aligned}$$

with $\text{Tr} = \text{Tr}_{\text{blunt}}$, operators normalised to $\lambda_{\max} = 1$, and the contrasts of the operators as parameters. Two observations motivate a different approach.

Sampling misses the extremes. For fixed X, Y the supremum in Exp. 1 is known exactly, $\sup_U z = \|YX\|_1 - \text{Tr}(XY) > 0$. Table 8 and Figure 2 show that the best of 20 000 Haar samples is already negative for $k \geq 4$ and moves further away from the supremum as k grows, while

batched gradient ascent (32 starts, 500 steps) finds it to a relative accuracy of 10^{-5} or better. This is concentration of measure: the unitaries close to the maximiser form a set whose Haar measure decays rapidly with the dimension. In particular, the absence of violations in Monte Carlo samples for $k = 16$ reported in [1] is a property of the sampling, not of the inequality.

k	violation found		$\sup z / \text{Tr } XY$	best sample $z / \text{Tr } XY$	ascent rel. error		time [s]	
	sampling	ascent			median	max	sampl.	ascent
2	2/4	4/4	4.4×10^{-5}	-0.00	5.4×10^{-8}	1.7×10^{-5}	0.015	0.86
4	0/4	4/4	2.0×10^{-3}	-0.19	9.4×10^{-8}	7.5×10^{-7}	0.049	0.86
8	0/4	4/4	3.2×10^{-3}	-0.58	5.0×10^{-7}	2.1×10^{-6}	0.18	1.2
16	0/4	4/4	4.5×10^{-3}	-0.78	2.6×10^{-6}	4.4×10^{-6}	0.77	2.6

Table 8: Exp. 1 for fixed random X, Y : Haar sampling versus gradient ascent (medians over 4 pairs; quick run on CPU (4 threads)).

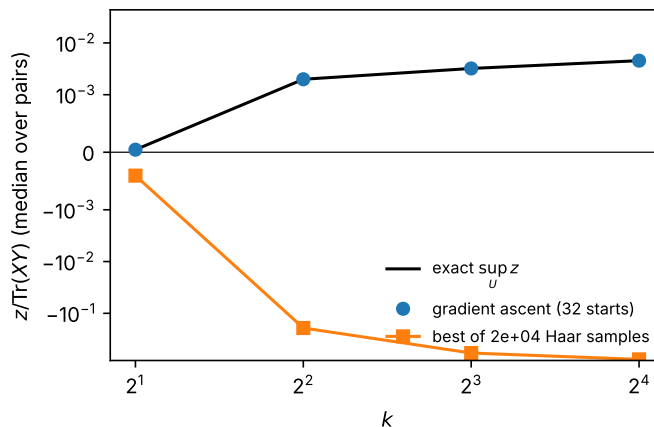


Figure 2: Best z found by sampling and by gradient ascent, relative to $\text{Tr}(XY)$, against the exact supremum (symmetric log scale).

Constrained optimisation. The natural objects are the envelopes

$$\bar{z}(\delta) = \sup\{z : \Delta(X) = \delta\}, \quad \underline{z}(\delta) = \inf\{z : \Delta(X) = \delta\},$$

(and the same with $\Delta(Y)$, or with both contrasts prescribed), the supremum being taken over all remaining variables. The module `optimize` parametrises the constraint set exactly: the spectrum is $\lambda_{\max} = 1$, $\lambda_{\min} = (1 - \delta)/(1 + \delta)$ and the other eigenvalues in between (through a sigmoid), the eigenvectors are $U = U_0 e^{G - G^*}$ with a Haar-random U_0 per start and a chart that is re-centred after every round, and for Exp. 3 the signs of the eigenvalues of X are drawn per start and kept fixed. Every grid point and every start is one element of the batch, so the whole envelope is computed in a single batched optimisation.

Figures 3 and 4 and Table 9 show the result. Every Monte Carlo sample of [1] lies between the computed envelopes; samples that fall outside the linear interpolation between grid points are re-optimised at their exact contrast, after which none remains outside. The envelopes are much wider than the sample clouds, already for $k = 2$.

Two envelopes can be checked analytically. In Exp. 3, $z \geq 0$ always:

Proposition 1. For $X = X^*$ and $Y \geq 0$, $\text{Tr}|YXY| \leq \text{Tr}(Y|X|Y)$, with equality if Y commutes with X .

Proof. With $X = X_+ - X_-$, $|X| = X_+ + X_-$, the triangle inequality for the trace norm gives $\|YX_+Y - YX_-Y\|_1 \leq \text{Tr}(YX_+Y) + \text{Tr}(YX_-Y)$. If $XY = YX$, then $|YXY| = Y^2|X|$. $\square$

The computed lower envelope of Exp. 3 is zero to machine precision. In Exp. 1, the lower envelope for prescribed $\Delta(X) = \delta$ equals $-(kC - 2\delta/(1 + \delta))$ when all sectors have size $k \geq 2$: $\text{Tr}(XY) \leq \text{Tr} X \leq kC - 1 + \lambda_{\min}(X)$ with equality for $Y = 1$, while $|\text{Tr}(UYX)|$ can be made zero by a cyclic permutation of the eigenbasis of X_c in every sector. The optimiser reproduces this curve to a relative deviation of $8.1\text{e-}03$. The upper envelopes of all three experiments are, to our knowledge, not known in closed form; the computed curves are lower bounds for them (a local search) and suggest conjectures for sharp contrast-dependent inequalities, which we leave for future work.

Exp.	k	C	inf z	sup z	Monte Carlo range of z	outside
1	2	1	-1.994	+0.2436	$[-1.98, +0.225]$	0
1	2	2	-3.991	+0.4765	$[-3.39, +0.328]$	0
1	4	1	-3.994	+0.472	—	—
2	2	1	-0.9931	+0.9934	$[-0.921, +0.975]$	0
2	2	2	-1.956	+1.963	$[-1.43, +1.72]$	0
2	4	1	-1.961	+1.957	—	—
3	2	1	0	+0.9927	$[0, +0.946]$	0
3	2	2	0	+1.968	$[0, +1.39]$	0
3	4	1	0	+1.962	—	—

Table 9: Global extremes of z over all contrasts found by constrained optimisation, the range of the Monte Carlo samples, and the number of samples outside the envelopes after re-optimisation (12 starts per grid point, 240 steps; quick run on CPU (4 threads)).

7 Symmetric tensor products and spin chains

The basis of `SpinChain` is, by construction, that of the iterated fused product of single sites: the tests check that S_i^z built by the chain coincides with $S_i^z \otimes 1$ formed with `kron` in the fused product of the algebra of `SpinChain(L-1)` with a single site. Beyond that, integrable chains provide exact results for every part of the module (Table 11, Figure 5; script `scripts/paper/chains.py`):

- *Free fermions.* By the Jordan–Wigner transformation the open XX chain ($\Delta = 0$) is a free-fermion model with single-particle energies $\varepsilon_k = J \cos \frac{\pi k}{L+1}$, so the spectrum of sector N consists of all sums of N distinct ε_k . For $L = 10$ all 2^L levels agree to 8.0×10^{-15} .
- *Heisenberg ring.* Ground-state energies agree with the tabulated exact values to the nine digits of the table; E_0/L approaches the Bethe-ansatz value $\frac{1}{4} - \ln 2 \approx -0.4431$. Every level of sector $N < L/2$ reappears in sector $N + 1$ (SU(2) multiplets), up to 1.5×10^{-14} .
- *Dynamics.* The melting of a domain wall $|\uparrow \cdots \uparrow \downarrow \cdots \downarrow\rangle$ in the XX chain, evolved in the sector $N = L/2$ (dimension 924 for $L = 12$), reproduces the single-particle prediction $\langle n_i(t) \rangle = [e^{-iht}C(0)e^{iht}]_{ii}$ to 5.0×10^{-15} .
- *Entanglement.* The entropy of a block of n sites of the XX ground state, obtained with `reduced_state`, agrees with Peschel’s formula $S = -\sum_\nu [\nu \ln \nu + (1 - \nu) \ln(1 - \nu)]$ over the

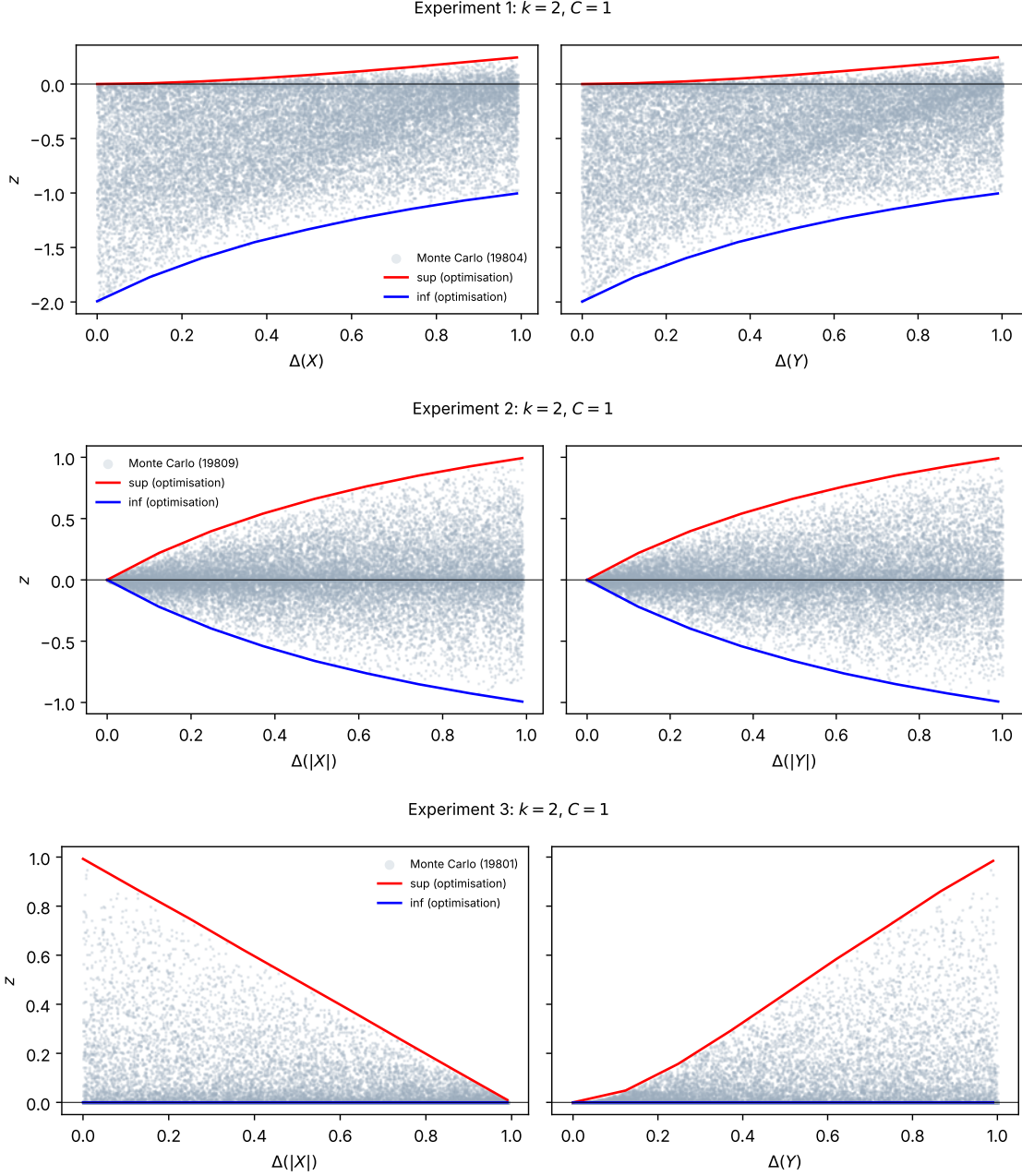


Figure 3: Sharp bounds versus sampling for the three trace functionals ($k = 2, C = 1$; one row per experiment). Horizontal axis: Michelson contrast of the first operator (left) or of the second operator (right); vertical axis: z for operators normalised to $\lambda_{\max} = 1$. Grey: the Monte Carlo samples of [1]; red and blue: the upper and lower envelopes found by constrained optimisation, the other contrast being free. The envelopes enclose every sample, and the samples do not reach them: in Exp. 1 the region $z > 0$ (violation of $|\text{Tr}(XUY)| \leq \text{Tr}(XY)$) is barely populated by samples but extends to $z \approx 0.24$; in Exp. 3 the lower envelope is the proven bound $z \geq 0$.

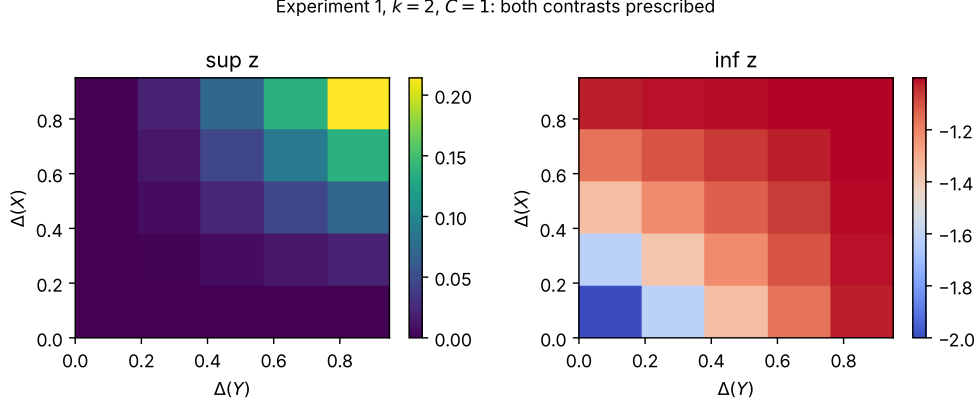


Figure 4: Exp. 1, $k = 2$, $C = 1$, both contrasts prescribed: $\sup z$ (left) and $\inf z$ (right) on a grid of $\Delta(Y)$ (horizontal) and $\Delta(X)$ (vertical). The violation $\sup z > 0$ needs both operators to be far from scalar and grows with both contrasts.

eigenvalues ν of the restricted correlation matrix [15], including the parity oscillations of open chains, to 3.3×10^{-12} .

- *Disorder.* For the Heisenberg chain with random fields $h_i \in [-W, W]$ the mean ratio of consecutive level spacings in the middle of the spectrum crosses over from the GOE value 0.5307 to the Poisson value $2 \ln 2 - 1 \approx 0.3863$ [16, 17] as W grows, the hallmark of many-body localisation. Here the batch dimension holds the disorder realisations (40 per point).
- *Momentum.* For periodic chains the basis $|a, k\rangle = N_a^{-1/2} \sum_j e^{-ikj} T^j |a\rangle$ over orbit representatives a of the translation T splits sector N into blocks of fixed momentum $k = 2\pi m/L$ (for $L = 12$ and $N = L/2$ of sizes 80, 75, 78, 76, 78, 75, 80, 75, 78, 76, 78, 75); the union of their spectra is the spectrum of the sector to 3.8×10^{-14} , and the ground state of the Heisenberg ring has $k = 0$ for even and $k = \pi$ for odd $L/2$ (Marshall’s sign rule). Momentum blocks are complex even for real Hamiltonians, one of the cases of Section 3 where the field is fixed by the problem.
- *Beyond dense matrices.* Lanczos with full re-orthogonalisation and restarts, batched over Hamiltonians, reproduces the free-fermion ground-state energies of open XX chains and the Heisenberg ring in sectors far beyond dense diagonalisation (Table 10); Krylov evolution of the domain wall for $L = 16$ (sector dimension 12870) agrees with the single-particle solution to 1.1×10^{-14} .
- *Open chains.* With jump operators S_i^- on every site (Lindblad terms between sectors $N \rightarrow N - 1$) and any N -conserving Hamiltonian the particle number performs a pure death process, so the sector distribution must be binomial, $p_n(t) = \binom{N_0}{n} e^{-n\gamma t} (1 - e^{-\gamma t})^{N_0 - n}$; the library reproduces it to 2.4×10^{-10} .

8 Comparison with NumPy/SciPy

To separate what the library adds from what plain array code already gives, the same computations were written three ways (`scripts/paper/baselines.py`; Table 12). (A) The Monte Carlo of Exp. 1: B pairs of positive operators with prescribed contrast and Haar-orthogonal eigenvectors in every

model	sector dim	E_0 (Lanczos)	E_0 (exact)	difference	time [s]
XX open, $L = 12$	924	-3.6481149053	-3.6481149053	8.9×10^{-16}	0.034
XX open, $L = 16$	12870	-4.9189757237	-4.9189757237	3.6×10^{-15}	0.31
XX open, $L = 18$	48620	-5.5547827927	-5.5547827927	8.9×10^{-16}	1.4
Heisenberg ring, $L = 16$	12870	-7.1422963606	-7.1422963610	3.8×10^{-10}	0.38

Table 10: Batched Lanczos on sparse sector Hamiltonians: ground-state energies against free fermions ($\sum$ of the $L/2$ lowest $\cos \frac{\pi k}{L+1}$) and the tabulated Heisenberg value (nine digits); quick run on CPU (4 threads).

L	dim ($N = L/2$)	E_0 (library)	E_0 (exact)	difference	E_0/L
4	6	-2.0000000000	-2.0000000000	1.0×10^{-16}	-0.50000
6	20	-2.802775638	-2.802775638	2.7×10^{-10}	-0.46713
8	70	-3.651093409	-3.651093409	6.3×10^{-11}	-0.45639
10	252	-4.515446354	-4.515446354	4.9×10^{-10}	-0.45154
12	924	-5.387390917	-5.387390917	4.5×10^{-10}	-0.44895

Table 11: Ground-state energy of the Heisenberg ring $H = \sum_i \mathbf{S}_i \cdot \mathbf{S}_{i+1}$; exact values to nine digits (quick run on CPU (4 threads)).

sector, a Haar-orthogonal $U \in \mathcal{M}$, and z . The direct NumPy/SciPy formulation loops over samples and sectors with `scipy.stats.ortho_group`; a hand-vectorised NumPy version uses stacked QR decompositions with the phase correction; the library needs three calls. (B) Ground states of sector Hamiltonians: `scipy.sparse.linalg.eigsh` (ARPACK [36]), one call per Hamiltonian, against the batched Lanczos solver. All results agree with each other.

The comparison is instructive in both directions. On a CPU the library is 60–600 times faster than the loop, but only on par with carefully vectorised NumPy (1–1.5 times faster): its value there is that the vectorised, correctly sampled, block-aware version is the default rather than an expert rewrite, and that the same code runs on a GPU and can be differentiated. For a single sparse eigenproblem on a CPU, ARPACK is faster than our Lanczos (which re-orthogonalises fully for robustness); the library’s solver pays off for batches of Hamiltonians (disorder averages) and on GPUs. We therefore recommend SciPy for single large eigenproblems on a CPU.

C	k	NumPy loop	NumPy vectorised	library (CPU)	library (GPU)
1	2	104	1.79	1.14	–
8	16	1.02e+03	713	448	–

L	sector dim	Hamiltonians	eigsh [s]	library [s]	$\max \Delta E_0 $
14	3432	1	0.00741	0.024	4.4e-15
14	3432	10	0.075	0.125	2.5e-14
16	12870	1	0.0274	0.0476	8.9e-16

Table 12: Top: time per Monte Carlo sample [μ s] for (A); bottom: ground states of Heisenberg rings (one Hamiltonian) and random-field chains ($W = 3$, several Hamiltonians) in the sector $N = L/2$ (quick run on CPU (4 threads)).

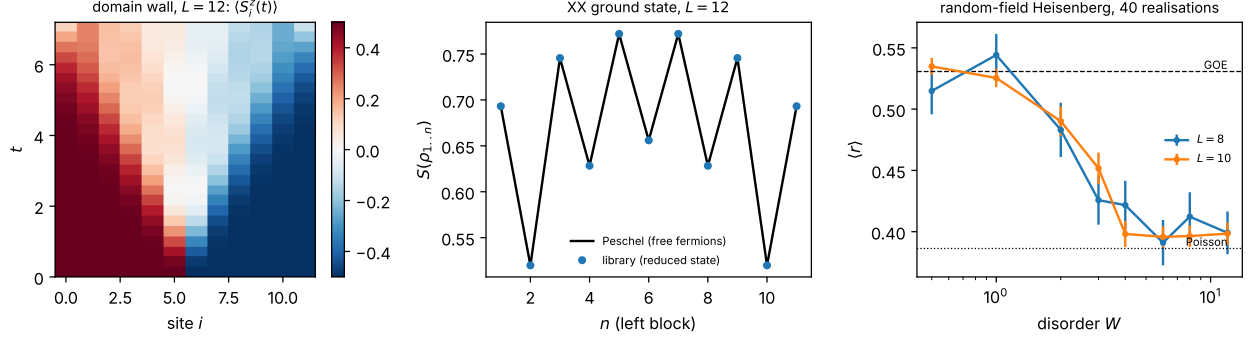


Figure 5: Left: $\langle S_i^z(t) \rangle$ after a domain-wall quench of the XX chain (light cone). Middle: entanglement entropy of the XX ground state, library vs. Peschel’s formula. Right: mean level-spacing ratio of the random-field Heisenberg chain, with the GOE and Poisson values.

9 Performance

Table 13 and Figure 6 report the wall-clock time per sample of library operations (not of isolated kernels), in single precision, with the batch size chosen so that the batch holds about 2^{22} matrix entries (quick run on CPU (4 threads); CPU with 4 threads). Expected behaviour: for single small blocks the CPU is faster, because kernel launches dominate; the GPU gains for many sectors and for the matrix products in channels and in the optimisation step; batched symmetric eigen- and singular-value solvers lose much of their advantage from $k \approx 64$ on, where the GPU libraries switch from batched small-matrix kernels to general ones.

The first version of this work reported GPU speedups for raw kernels on a P100 and found them only for part of the configurations. That observation reflects where GPUs help: a GPU needs enough independent work per kernel launch. For computations in (1) this means many samples, many sectors or large blocks, and matrix products rather than decompositions of small matrices. The design follows from it: everything is batched, so that the batch dimension supplies the parallel work, and the cost module warns before configurations that would be slow. For workloads that are small or dominated by a single eigenproblem, the CPU (or SciPy, Section 8) is the right tool, and the library runs there unchanged.

10 Limitations and roadmap

- *Layout.* Blocks are padded to $k_{\max}$. For very different sector sizes (binomial sectors of spin chains) one works in single sectors; operators acting on all sectors at once would need a bucketed layout, which is not implemented yet.
- *Symmetries.* Abelian charges with an arbitrary fusion rule and lattice translations are supported; non-abelian symmetries (SU(2) multiplets, Clebsch–Gordan structure), reflections and spin flip are not used to reduce blocks further. Dense methods reach about 16–18 spins, the sparse Lanczos and Krylov solvers about 24–26 spins per sector (memory of the Krylov basis); tensor-network methods are out of scope.
- *Optimisation.* The search is local; envelopes are lower bounds for suprema (upper bounds for infima). Gradients of spectral functions are singular at degenerate spectra; the parametrisations avoid this for the contrast constraint, but not for every user-defined objective.

operation	k	C	batch	time per sample [μ s]
generate	4	1	4096	1.33
f(A) eigh	4	1	4096	3.19
A SVD	4	1	4096	6.14
contrast	4	1	4096	2.07
channel	4	1	4096	0.595
exp(tL)	4	1	4096	59.6
opt. step	4	1	4096	13.6
generate	4	16	1024	28.1
f(A) eigh	4	16	1024	44.1
A SVD	4	16	1024	90
contrast	4	16	1024	24.3
channel	4	16	1024	7.81
exp(tL)	4	16	1024	1.42e+03
opt. step	4	16	1024	314
generate	16	1	1024	18.1
f(A) eigh	16	1	1024	35.8
A SVD	16	1	1024	67.2
contrast	16	1	1024	14
channel	16	1	1024	2.96
exp(tL)	16	1	1024	1.67e+04
opt. step	16	1	1024	205
generate	16	16	64	439
f(A) eigh	16	16	64	495
A SVD	16	16	64	938
contrast	16	16	64	200
channel	16	16	64	46.7
exp(tL)	16	16	64	2.48e+05
opt. step	16	16	64	3e+03

Table 13: Wall-clock time per sample of library operations for C sectors of size k and the batch size shown (single precision; “exp(tL)” uses a smaller batch because its superoperators have k^4 entries). With a GPU the table also gives the CPU time and the speedup CPU/GPU.

- *Scale.* Choi matrices, superoperators and $e^{t\mathcal{L}}$ work with $k_c^2 \times k_c^2$ matrices and are limited to blocks of a few tens; RK4 integration scales to larger blocks.
- *Scope.* Finite dimensions and Type I only; no tensor-network or sparse methods.

11 Availability and reproducibility

The code (MIT licence) is available at https://github.com/andre-a-no/von_neumann_type_i. `bash scripts/paper/run_all.sh full cuda` regenerates every table and figure of this text on a GPU; `bash scripts/paper/run_all.sh` runs a reduced version on a CPU in a few minutes. The tables state on which device and in which mode they were produced.

References

- [1] I. Nikolaeva, A. Novikov, “Finite-dimensional Type I von Neumann algebras in PyTorch: a GPU-accelerated framework for random block-diagonal operators”, arXiv:2606.15882v1 (2026).

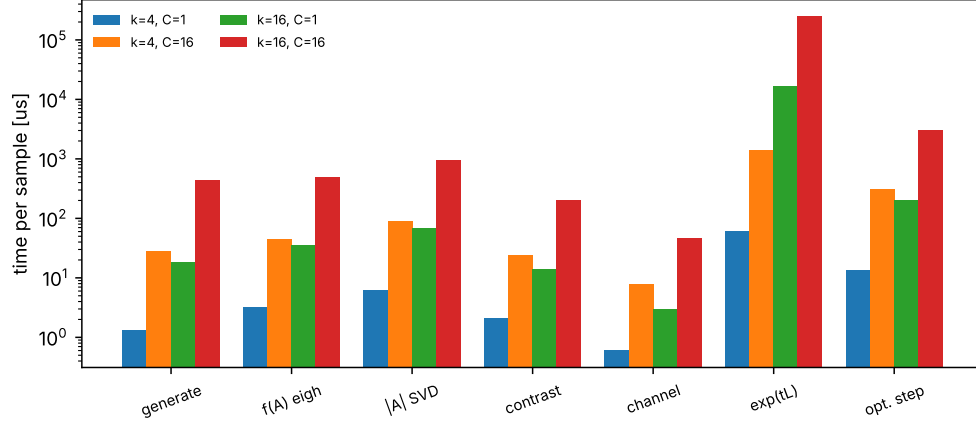


Figure 6: Time per sample (logarithmic axis) of the operations of Table 13, one bar per configuration (k, C) .

- [2] L. T. Gardner, “An inequality characterizes the trace”, *Canad. J. Math.* 31 (1979), 1322–1328.
- [3] D. Petz, J. Zemánek, “Characterizations of the trace”, *Linear Algebra Appl.* 111 (1988), 43–52.
- [4] A. Novikov, O. Tikhonov, “Characterization of central elements of operator algebras by inequalities”, *arXiv:1502.01267* (2015).
- [5] D. Virosztek, “Connections between centrality and local monotonicity of certain functions”, *J. Math. Anal. Appl.* 453 (2017), 221–226.
- [6] S. A. Abed, I. A. Nikolaeva, A. A. Novikov, “Generalisation of Michelson contrast for operators and its properties”, *Lobachevskii J. Math.* 45 (2024), 3835–3848.
- [7] A. Novikov, S. A. Abed, I. Nikolaeva, “Generalisation of Michelson contrast for operators”, *arXiv:2010.10130* (2020).
- [8] J. R. Johansson, P. D. Nation, F. Nori, “QuTiP 2: A Python framework for the dynamics of open quantum systems”, *Comput. Phys. Commun.* 184 (2013), 1234–1240.
- [9] Qiskit contributors, “Qiskit: An open-source framework for quantum computing”, 2019, <https://qiskit.org>.
- [10] P. Guilmin, R. Gautier, A. Bocquet, E. Genois, D. Weiss, “dynamiqs: an open-source Python library for GPU-accelerated and differentiable simulation of quantum systems”, <https://github.com/dynamiqs/dynamiqs>.
- [11] F. Mezzadri, “How to generate random matrices from the classical compact groups”, *Notices Amer. Math. Soc.* 54 (2007), 592–604.
- [12] M. L. Mehta, *Random Matrices*, 3rd ed., Elsevier, 2004.
- [13] D. N. Page, “Average entropy of a subsystem”, *Phys. Rev. Lett.* 71 (1993), 1291–1294.
- [14] K. Życzkowski, H.-J. Sommers, “Induced measures in the space of mixed quantum states”, *J. Phys. A* 34 (2001), 7111–7125.

- [15] I. Peschel, “Calculation of reduced density matrices from correlation functions”, *J. Phys. A* 36 (2003), L205–L208.
- [16] V. Oganesyan, D. A. Huse, “Localization of interacting fermions at high temperature”, *Phys. Rev. B* 75 (2007), 155111.
- [17] Y. Y. Atas, E. Bogomolny, O. Giraud, G. Roux, “Distribution of the ratio of consecutive level spacings in random matrix ensembles”, *Phys. Rev. Lett.* 110 (2013), 084101.
- [18] M. Takesaki, *Theory of Operator Algebras I*, Springer, 2002.
- [19] G. C. Wick, A. S. Wightman, E. P. Wigner, “The intrinsic parity of elementary particles”, *Phys. Rev.* 88 (1952), 101–105.
- [20] S. D. Bartlett, T. Rudolph, R. W. Spekkens, “Reference frames, superselection rules, and quantum information”, *Rev. Mod. Phys.* 79 (2007), 555–609.
- [21] W. H. Zurek, “Decoherence, einselection, and the quantum origins of the classical”, *Rev. Mod. Phys.* 75 (2003), 715–775.
- [22] A. W. Sandvik, “Computational studies of quantum spin systems”, *AIP Conf. Proc.* 1297 (2010), 135–338.
- [23] P. Weinberg, M. Bukov, “QuSpin: a Python package for dynamics and exact diagonalisation of quantum many body systems part I: spin chains”, *SciPost Phys.* 2 (2017), 003.
- [24] M. Goldstein, E. Sela, “Symmetry-resolved entanglement in many-body systems”, *Phys. Rev. Lett.* 120 (2018), 200602.
- [25] J. C. Xavier, F. C. Alcaraz, G. Sierra, “Equipartition of the entanglement entropy”, *Phys. Rev. B* 98 (2018), 041106.
- [26] P. Zanardi, M. Rasetti, “Noiseless quantum codes”, *Phys. Rev. Lett.* 79 (1997), 3306–3309.
- [27] E. Knill, R. Laflamme, L. Viola, “Theory of quantum error correction for general noise”, *Phys. Rev. Lett.* 84 (2000), 2525–2528.
- [28] C. Bény, A. Kempf, D. W. Kribs, “Generalization of quantum error correction via the Heisenberg picture”, *Phys. Rev. Lett.* 98 (2007), 100502.
- [29] S. Hadfield, Z. Wang, B. O’Gorman, E. G. Rieffel, D. Venturelli, R. Biswas, “From the quantum approximate optimization algorithm to a quantum alternating operator ansatz”, *Algorithms* 12 (2019), 34.
- [30] F. J. Dyson, “The threefold way. Algebraic structure of symmetry groups and ensembles in quantum mechanics”, *J. Math. Phys.* 3 (1962), 1199–1215.
- [31] T. Cohen, M. Welling, “Group equivariant convolutional networks”, *Proc. ICML 2016*, PMLR 48, 2990–2999.
- [32] M. Finzi, M. Welling, A. G. Wilson, “A practical method for constructing equivariant multilayer perceptrons for arbitrary matrix groups”, *Proc. ICML 2021*, PMLR 139, 3318–3328.
- [33] C. H. Martin, M. W. Mahoney, “Implicit self-regularization in deep neural networks: evidence from random matrix theory and implications for learning”, *J. Mach. Learn. Res.* 22 (2021), 1–73.

- [34] C. R. Harris et al., “Array programming with NumPy”, *Nature* 585 (2020), 357–362.
- [35] P. Virtanen et al., “SciPy 1.0: fundamental algorithms for scientific computing in Python”, *Nat. Methods* 17 (2020), 261–272.
- [36] R. B. Lehoucq, D. C. Sorensen, C. Yang, *ARPACK Users’ Guide*, SIAM, 1998.
- [37] M. Fishman, S. R. White, E. M. Stoudenmire, “The ITensor software library for tensor network calculations”, *SciPost Phys. Codebases* 4 (2022).
- [38] J. Hauschild, F. Pollmann, “Efficient numerical simulations with tensor networks: Tensor Network Python (TeNPy)”, *SciPost Phys. Lect. Notes* 5 (2018).
- [39] M. Fukuda, R. König, I. Nechita, “RTNI – a symbolic integrator for Haar-random tensor networks”, *J. Phys. A* 52 (2019), 425303.
- [40] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library”, *NeurIPS* 2019.